



Command Line and Relative Paths

Instructor: Shaolin Xie



Overview

- You will learn to use basic terminal commands to navigate the filesystem (these commands work on Linux/Mac and some Windows machines)



Command Line

- Using the command line is an important skill to understand and navigate modern computer systems
- You can use the command line to more quickly and easily control file creation, location, and modification



Commands To Use Today

- `ls` and `ls -l` and `ls -a`: List the files (and details) in the current directory. The flag “a” shows both visible and hidden files and directories.
- `cd folder-path`: Change directory to folder-path (`cd ~` brings you to the home directory, `cd ..` goes up to the parent folder)
- `cp src.cpp dest`: Copy the source file to the destination file or folder
- `mv old.cpp new.cpp`: Move/rename old.cpp to new.cpp
- `rm filename` or `rm -r folder`: Remove (delete) filename or folder (requires the -r flag for recursive removal)
- `mkdir folder`: Make directory (create a folder)
- `pwd`: Shows your present working directory. Helpful in remembering where you are in the file system.
- `cat filename`: Shows the content of the file



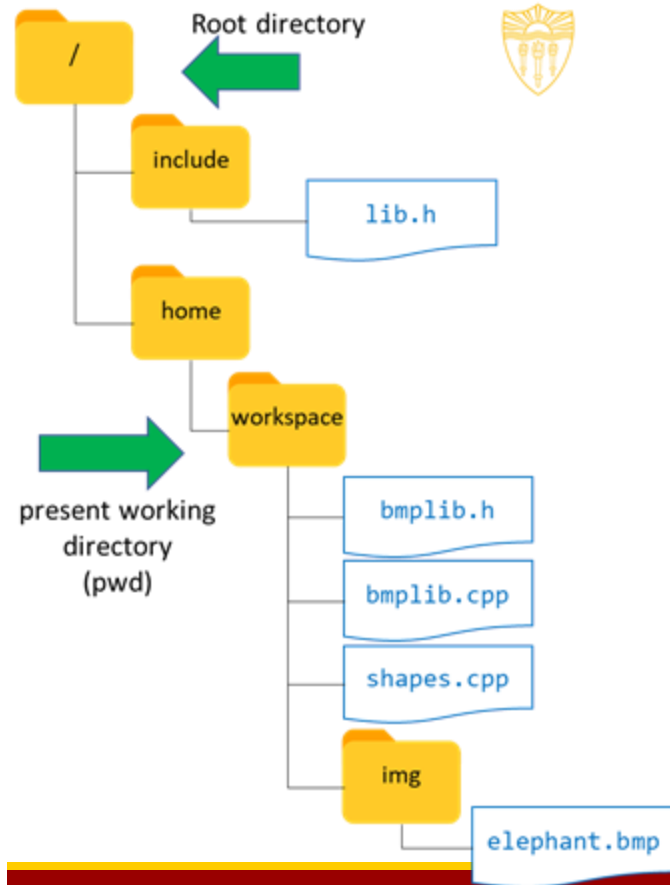
File Visibility

- Files and directories are hidden if their file name has a '.' as the first character of its name.
- Files and directories can be unhidden by renaming the file without the '.' at the front of the file name.

```
codio@margopanther-timeactor:~/workspace/temp$ ls
hello.txt
codio@margopanther-timeactor:~/workspace/temp$ ls -a
.  ..  hello.txt  .hidden.txt
```

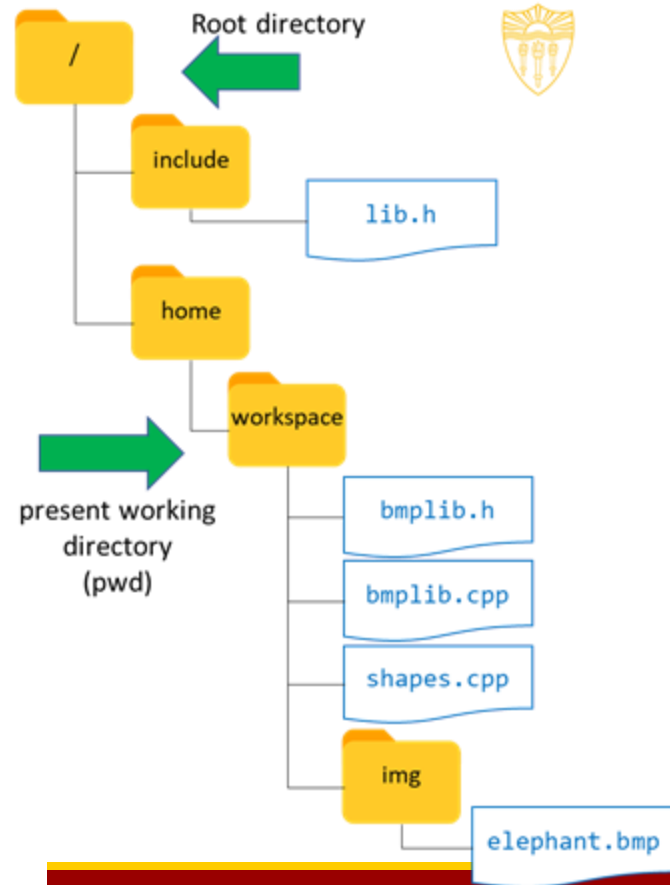
Absolute and Relative Paths

- An **absolute path** always starts its reference from the root folder (/)
 - Ex: `/home/workspace/img/elephant.bmp`
 - If the path starts with / the terminal interprets it as an absolute path
- A **relative path** implicitly starts from the present working directory
 - Ex: `img/elephant.bmp`
 - If the path does NOT start with / the terminal interprets it as a relative path
- You can use whatever path style is easier (usually relative paths save typing)



Path Shortcuts

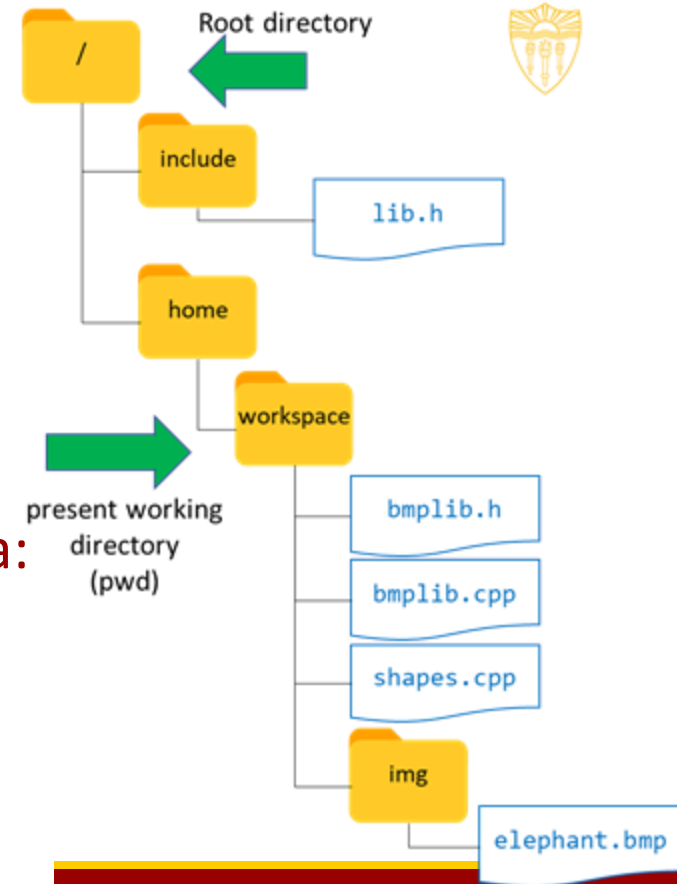
- There are a few "alias" or shortcut paths
- A single dot (.) is a placeholder for the present working directory
 - `cp /home/workspace/img/elephant.bmp .`
will copy elephant.bmp to the present working directory (i.e. /home/workspace)
- Two dots (..) is a placeholder for the parent folder (one level up from the present working directory)
 - `cd ..`
Will change the present working directory to the parent folder



Sample Commands [1]

- Suppose our present working directory is /home/workspace
- We could
 - Change directory to img via
`$ cd img`
 - Change directory to the root folder (/) via:
`$ cd ../../..` (relative path...OR...)
`$ cd /` (absolute path)
 - List all the files in folder 2 levels up via:
`$ ls ../../..`

Which might show:
`include home`

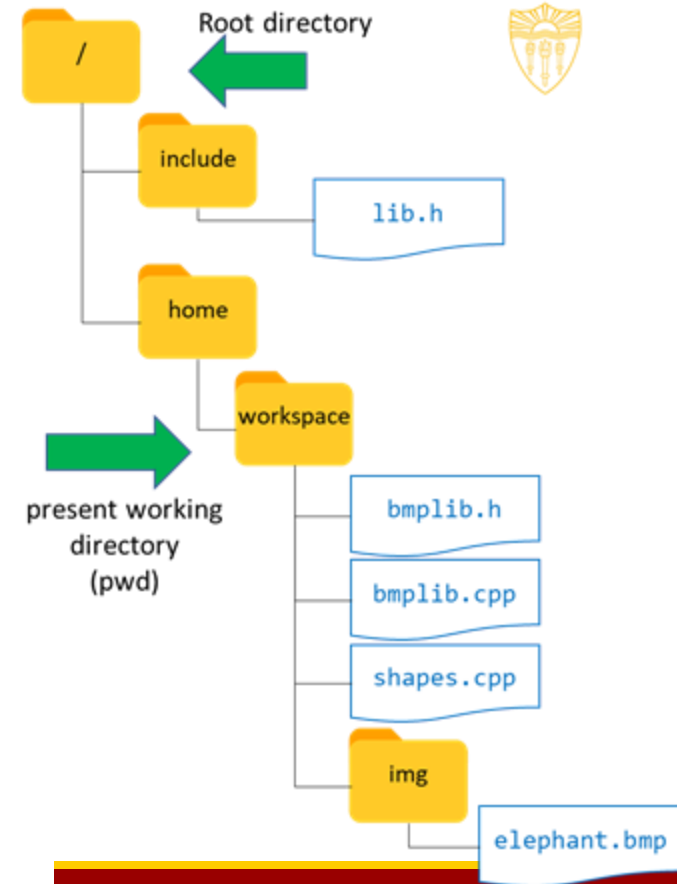


Sample Commands [2]

- Suppose our present working directory is /home/workspace
- We could
 - Print files/folders in the current folder with details
 - `$ ls -l`

Which might show:

```
codio@qualitysandra-respectchaos:~/workspace$ ls -l
total 40
-rw-r--r-- 1 codio codio 12822 Jul 26  2020 bmplib.cpp
-rw-r--r-- 1 codio codio 1024 Jul 16  2020 bmplib.h
-rw-r--r-- 1 codio codio 687 Sep 14 16:41 demo.cpp
-rw-rw-r-- 1 codio codio 1991 Sep 11 02:32 game.cpp
-rw-r--r-- 1 codio codio 595 Aug 17 16:33 Makefile
-rw-r--r-- 1 codio codio 795 Sep  2 16:31 shapes.cpp
-rw-rw-r-- 1 codio codio 537 Sep 11 01:36 stringdemo.cpp
codio@qualitysandra-respectchaos:~/workspace$
```



Sample Commands [3]

- Suppose our present working directory is /home/workspace
- We could
 - Copy lib.h from the /include folder to your current folder:
 - `$ cp ../../include/lib.h .`
 - Move ALL files ending in .cpp to the img folder:
 - `$ mv *.cpp img/`
 - Remove the elephant.bmp file
 - `$ rm img/elephant.bmp`

